Martin Marietta Astronautics Group
MS L3021
P. O. Box 179
Denver, CO 80201
1987 Sep 09

Robert C. Ashenhurst
ACM Forum Editor
University of Chicago
1101 E. 58th St.
Chicago, IL 60637

Dear Sir:

We read Nielsen and Shumate's article [1] with interest. Their approach is very similar to our work on Ada™ software development techniques. Having worked on the Remote Data Acquisition System (RDAS) problem presented by Cherry [2], which is very similar to their Remote Temperature Sensor (RTS) problem, we have noted some problems with their solution which decrease the quality of the solution, and so make their Ada Design Methodology (ADM) seem of lower quality than it could be. Most of these problems stem, in our opinion, from the fact that ADM makes encapsulation decisions late in the development process. We have found that considering encapsulation from the beginning avoids many of these problems.

We were surprised, considering the number of references to time in the problem specification, that the clock device did not appear on the Context Diagram (Figure 4). In our opinion, there are three entities in the RTS external environment: the communication hardware, the digital thermometer hardware, and the system clock.

Some aspects of a good Ada design include reusability and encapsulation. Reusability means the design should make use of existing components when possible, and identify and create new reusable components if they do not already exist. ADM has identified the reusability of standard data structures and tasking paradigms (queues and forwarders) but has ignored the fact that hardware is reused. We also make use of the edges-in approach, but we further propose that each edge entity should produce or make use of a hardware-dependent, application-independent reusable interface component. For example, other applications may make use of the same Digital Thermometer (DT) as RTS, but not need the communication system. If a reusable interface to this component doesn't exist, the developers of RTS should create it. The same applies to the communication system and the clock. Luckily the reusable interface to the clock already exists: It is the standard package CALENDAR, which is used by RTS.

Encapsulation means the creation of loosely-coupled, highly-cohesive modules which abstract features of the problem and hide unnecessary information about the implementation of the abstraction. A well-encapsulated design should limit the effect of changes to the system to as few modules as possible. For example, an obvious possible change to RTS would be the replacement of the DT with another model. This new DT may just be more dependable than the old one, or it may provide a different range of furnaces or a different accuracy of temperatures.

It is educational to consider the effects of changing the DT on the ADM solution to RTS. Assume that the new DT can handle twice as many furnaces as the old one (thirty-two vs. sixteen) numbered from 1 to 32, and provides temperatures with an accuracy of one-tenth of a degree from -10,000.0 to 10,000°C. It uses different addresses than the old DT and does not generate an interrupt. Instead it guarantees that the temperature will be found at the correct location no less than 0.01 seconds after it receives the furnace number. The host computer will continue to use

only sixteen furnaces numbered from 0 to 15, and will still expect temperatures accurate to one degree from 0 to 1,000°C.

The types of the furnace numbers and temperature reading returned by the DT are in the specification of package DEFINITIONS. The addresses used by the DT are in the specification of package HDP. Clearly both of these must be changed. DEFINITIONS is used by the specification of DEVICE_HANDLERS, HOST_INPUT, TEMP_READING, and HOST_OUTPUT. HDP is used by the body of DEVICE_HANDLERS, which must be changed because it makes reference to the interrupt address, which is no longer used. Of course the body of DT_HANDLER must be changed, but in addition further changes must be made throughout the system to accommodate the conversion from one numbering system to another and from one representation of temperature to another. Every compilation unit in the system must be recompiled. In a truly large system this compilation alone could take hours.

Contrast this to a properly encapsulated, reusable DT package. The package would be replaced by the corresponding package for the new DT, which would provide an identical interface except for the types. The module which makes use of the DT interface would be modified to incorporate a converter routine which it would use to access the DT. Since this module would also perform the buffering of readings and provide readings to the transmission system when required in ASCII format, none of the rest of the system would be changed. Since the changes to this module are to the body, not the specification, none of the rest of the system would need to be recompiled.

It is gratifying to see Ada's uniqueness for real-time systems recognized in the general literature. The problems we have identified with ADM invalidate the claim that ADM "takes advantage of Ada's excellent design features [1]." We offer this response so that potential Ada users might not conclude that the encapsulation shown in the solution to the RTS problem is the correct way to use Ada packages. The purpose of the ACM Forum is to provide a dialogue through which problems such as this can be identified and resolved. In the end we should all gain from the process.

REFERENCES

1. Nielsen, Kjell W., and Ken Shumate, "Designing Large Real-Time Systems with Ada," Commun. ACM, vol. 30, no. 8 (1987 Aug).

2. Cherry, George W., PAMELA Designers Handbook, The Analytic Sciences Corporation, 1986.

Jeffrey R. Carter

Charles D. McKeen